

LABSHEET - 7

HIMANI CHAUDHARY

BTECH(CSE) 3RD YEAR

CU240250967

SECTION - B

ROLL NO. - 24

Mastery in Data Analytics and Visualizations and Career Advancement (Technical)

```
# EXPERIMENT NO. 7
# CUSTOMER SEGMENTATION USING K-MEANS CLUSTERING
# =====

# Import libraries
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
# 1. LOAD DATASET
# =====

file_path = "/content/WA_Fn-UseC_-Telco-Customer-Churn.csv"
# Check whether file exists
if not os.path.exists(file_path):
    print("ERROR: Dataset file not found!")
    print("Please check this path:")
    print(file_path)
    exit()

# Read dataset
df = pd.read_csv(file_path)

print("\n" + "=" * 60)
print("DATASET")
print("=" * 60)

print("HIMANI CHAUDHARY")
print("\nFirst 5 rows:")
print(df.head())
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
DATASET
=====
HIMANI CHAUDHARY

First 5 rows:
  customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  7590-VHVEG  Female                0      Yes          No        1            No
1  5575-GNVDE   Male                0      No           No       34            Yes
2  3668-QPYBK   Male                0      No           No        2            Yes
3  7795-CFOCW   Male                0      No           No       45            No
4  9237-HQITU   Female              0      No           No        2            Yes

  MultipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \
0  No phone service            DSL              No  ...              No
1                No            DSL              Yes  ...              Yes
2                No            DSL              Yes  ...              No
3  No phone service            DSL              Yes  ...              Yes
4                No  Fiber optic              No  ...              No

  TechSupport  StreamingTV  StreamingMovies  Contract  PaperlessBilling  \
0          No          No          No  Month-to-month          Yes
1          No          No          No    One year              No
2          No          No          No  Month-to-month          Yes
3          Yes          No          No    One year              No
4          No          No          No  Month-to-month          Yes
```

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]
Himani Chaudhary
CU240250967

2. DATASET INFORMATION

=====

```
print("\n" + "=" * 60)
print("DATASET INFORMATION")
print("=" * 60)
```

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

```
print("\nColumn Names:")
print(df.columns.tolist())
```

```
print("\nData Types:")
print(df.dtypes)
```

```
print("\nMissing Values:")
print(df.isnull().sum())
print("Himani Chaudhary")
print("CU240250967")
```

DATASET INFORMATION

=====

Rows: 7043
Columns: 21

Column Names:

['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetSer

Data Types:

```
customerID      object
gender          object
SeniorCitizen   int64
Partner         object
Dependents      object
tenure          int64
PhoneService    object
MultipleLines    object
InternetService object
OnlineSecurity  object
OnlineBackup    object
DeviceProtection object
TechSupport     object
StreamingTV     object
StreamingMovies object
Contract        object
PaperlessBilling object
PaymentMethod   object
MonthlyCharges  float64
TotalCharges    object
Churn           object
dtype: object
```

Missing Values:

```
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines    0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
```

```
dtype: int64
Himani Chaudhary
CU240250967
```

```
# 3. DATA PREPROCESSING
# =====

# Remove duplicate rows
df = df.drop_duplicates()

# Convert TotalCharges to numeric
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(
        df["TotalCharges"],
        errors="coerce"
    )

# Fill missing numeric values with mean
numeric_columns = df.select_dtypes(
    include=np.number
).columns

for column in numeric_columns:
    df[column] = df[column].fillna(
        df[column].mean()
    )

print("\n" + "=" * 60)
print("AFTER PREPROCESSING")
print("=" * 60)

print("Dataset shape:", df.shape)

print("\nMissing values after preprocessing:")
print(df.isnull().sum())
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
AFTER PREPROCESSING
=====
Dataset shape: (7043, 21)

Missing values after preprocessing:
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
Himani Chaudhary
CU240250967
```

```
# 4. CHECK REQUIRED COLUMNS
# =====

required_columns = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

for column in required_columns:

    if column not in df.columns:
```

```

print("\nERROR: Required column not found:")
print(column)

print("\nAvailable columns:")
print(df.columns.tolist())

exit()
print("Himani Chaudhary")
print("CU240250967")

```

```

# 5. SELECT FEATURES
# =====

features = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

X = df[features].copy()

print("\n" + "=" * 60)
print("SELECTED FEATURES")
print("=" * 60)

print(features)

print("\nFirst 5 records:")
print(X.head())
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
SELECTED FEATURES
=====
['tenure', 'MonthlyCharges', 'TotalCharges']

First 5 records:
   tenure  MonthlyCharges  TotalCharges
0        1           29.85           29.85
1       34           56.95          1889.50
2         2           53.85           108.15
3       45           42.30          1840.75
4         2           70.70           151.65
Himani Chaudhary
CU240250967

```

```

# 6. FEATURE SCALING
# =====

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\n" + "=" * 60)
print("SCALED DATA")
print("=" * 60)

print(X_scaled[:5])
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
SCALED DATA
=====
[[-1.27744458 -1.16032292 -0.99497138]
 [ 0.06632742 -0.25962894 -0.17387565]
 [-1.23672422 -0.36266036 -0.96039939]
 [ 0.51425142 -0.74653546 -0.19540036]
 [-1.23672422  0.19736523 -0.94119274]]
Himani Chaudhary
CU240250967

```

```

# 7. ELBOW METHOD
# =====

inertia = []

```

```

K_range = range(1, 11)

for k in K_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    inertia.append(
        kmeans.inertia_
    )

# Display Elbow graph
plt.figure(figsize=(8, 5))

plt.plot(
    K_range,
    inertia,
    marker="o",
    color="blue"
)

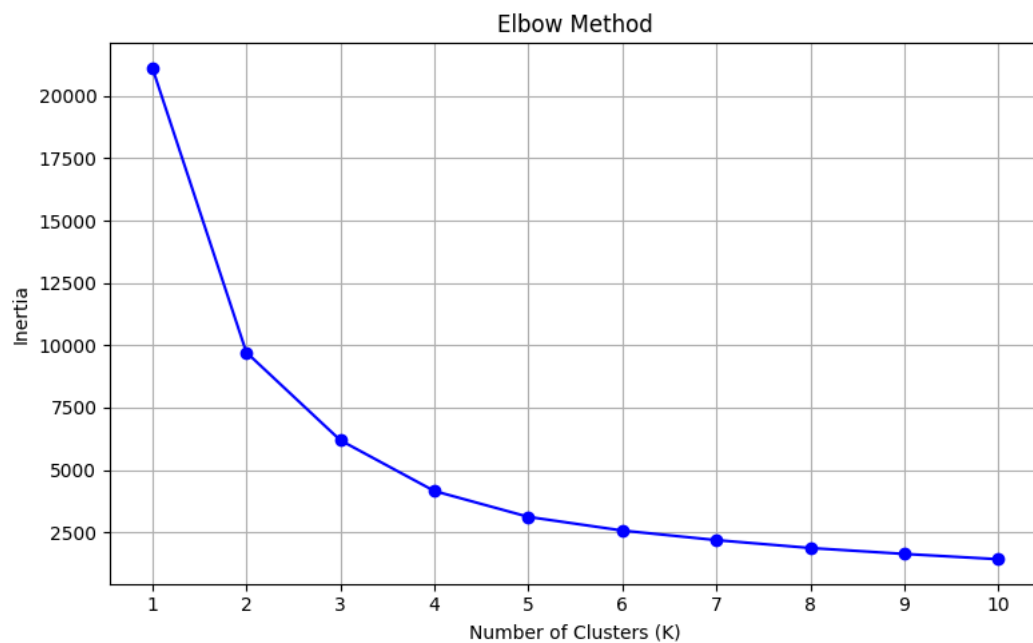
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(K_range)

plt.grid(True)

plt.tight_layout()

plt.show()
print("Himani Chaudhary")
print("CU240250967")

```



Himani Chaudhary
CU240250967

```

# 8. SILHOUETTE SCORE
# =====

silhouette_scores = []

K_range_silhouette = range(2, 11)

print("\n" + "=" * 60)
print("SILHOUETTE SCORES")
print("=" * 60)

print("HIMANI CHAUDHARY")

```

```
for k in K_range_silhouette:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(
        X_scaled,
        labels
    )

    silhouette_scores.append(score)

    print(
        "K =", k,
        "Silhouette Score =",
        round(score, 4)
    )

# Display Silhouette graph
plt.figure(figsize=(8, 5))

plt.plot(
    K_range_silhouette,
    silhouette_scores,
    marker="o",
    color="green"
)

plt.title("Silhouette Score")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")

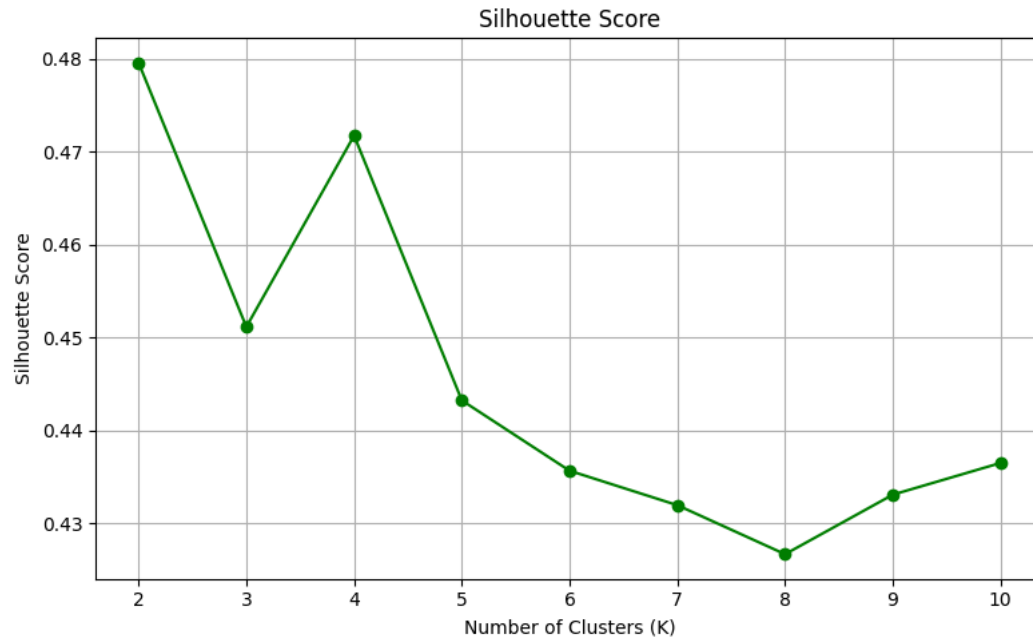
plt.xticks(K_range_silhouette)

plt.grid(True)

plt.tight_layout()

plt.show()
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
SILHOUETTE SCORES
=====
HIMANI CHAUDHARY
K = 2 Silhouette Score = 0.4796
K = 3 Silhouette Score = 0.4511
K = 4 Silhouette Score = 0.4717
K = 5 Silhouette Score = 0.4432
K = 6 Silhouette Score = 0.4356
K = 7 Silhouette Score = 0.4319
K = 8 Silhouette Score = 0.4267
K = 9 Silhouette Score = 0.4331
K = 10 Silhouette Score = 0.4365
```



Himani Chaudhary
CU240250967

```
# 9. SELECT OPTIMAL K
# =====

best_k = K_range_silhouette[
    np.argmax(silhouette_scores)
]

print("\n" + "=" * 60)
print("OPTIMAL NUMBER OF CLUSTERS")
print("=" * 60)

print("Optimal Number of Clusters:", best_k)

print("Himani Chaudhary")
print("CU240250967")
```

```
=====
OPTIMAL NUMBER OF CLUSTERS
=====
Optimal Number of Clusters: 2
Himani Chaudhary
CU240250967
```

```
# 10. APPLY K-MEANS CLUSTERING
# =====

kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(
    X_scaled
)

print("Himani Chaudhary")
print("CU240250967")
```

Himani Chaudhary
CU240250967

```
# 11. CLUSTER CENTROIDS
# =====

centroids_scaled = kmeans.cluster_centers_

centroids = scaler.inverse_transform(
    centroids_scaled
)

centroid_df = pd.DataFrame(
    centroids,
    columns=features
)

print("\n" + "=" * 60)
print("CLUSTER CENTROIDS")
print("=" * 60)

print(centroid_df)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CLUSTER CENTROIDS
=====
      tenure  MonthlyCharges  TotalCharges
0  20.026057         52.155842       871.038831
1  56.852181         89.759826      5083.897162
Himani Chaudhary
CU240250967
```

```
# 12. DISPLAY CUSTOMER CLUSTERS
# =====

print("\n" + "=" * 60)
print("CUSTOMER CLUSTERS")
print("=" * 60)

display_columns = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges",
    "Cluster"
]

# Add CustomerID if available
if "customerID" in df.columns:

    display_columns.insert(
        0,
        "customerID"
    )

# Add gender if available
if "gender" in df.columns:

    display_columns.insert(
        1,
        "gender"
    )

print(
    df[display_columns].head(20)
)

print("\nHIMANI CHAUDHARY")
print("CU240250967")
```

```
=====
CUSTOMER CLUSTERS
=====
   customerID  gender  tenure  MonthlyCharges  TotalCharges  Cluster
0  7590-VHVEG  Female      1           29.85           29.85        0
1  5575-GNVDE   Male     34           56.95          1889.50        0
2  3668-QPYBK   Male      2           53.85           108.15        0
3  7795-CFOCW   Male     45           42.30          1840.75        0
```

4	9237-HQITU	Female	2	70.70	151.65	0
5	9305-CDSKC	Female	8	99.65	820.50	0
6	1452-KIOVK	Male	22	89.10	1949.40	0
7	6713-OKOMC	Female	10	29.75	301.90	0
8	7892-POOKP	Female	28	104.80	3046.05	1
9	6388-TABGU	Male	62	56.15	3487.95	1
10	9763-GRSKD	Male	13	49.95	587.45	0
11	7469-LKBCI	Male	16	18.95	326.80	0
12	8091-TTVAX	Male	58	100.35	5681.10	1
13	0280-XJGEX	Male	49	103.70	5036.30	1
14	5129-JLPIS	Male	25	105.50	2686.05	1
15	3655-SNQYZ	Female	69	113.25	7895.15	1
16	8191-XWSZG	Female	52	20.65	1022.95	0
17	9959-WOFKT	Male	71	106.70	7382.25	1
18	4190-MFLUW	Female	10	55.20	528.35	0
19	4183-MYFRB	Female	21	90.05	1862.90	0

HIMANI CHAUDHARY
CU240250967

```
# 13. CLUSTER SIZE
# =====

print("\n" + "=" * 60)
print("CLUSTER SIZE")
print("=" * 60)

cluster_counts = (
    df["Cluster"]
    .value_counts()
    .sort_index()
)

print(cluster_counts)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CLUSTER SIZE
=====
Cluster
0    4683
1     2360
Name: count, dtype: int64
Himani Chaudhary
CU240250967
```

```
# 14. CLUSTER ANALYSIS
# =====

cluster_summary = (
    df.groupby("Cluster")[
        [
            "tenure",
            "MonthlyCharges",
            "TotalCharges"
        ]
    ].mean()
)

print("\n" + "=" * 60)
print("CLUSTER SUMMARY")
print("=" * 60)

print(cluster_summary)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CLUSTER SUMMARY
=====
      tenure  MonthlyCharges  TotalCharges
Cluster
0    20.027333      52.165706      871.389943
1     56.865254      89.756186     5084.985551
Himani Chaudhary
CU240250967
```

```
# 15. VISUALIZE CLUSTERS
# =====

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x="MonthlyCharges",
    y="TotalCharges",
    hue="Cluster",
    palette="viridis",
    s=80
)

# Plot cluster centroids
plt.scatter(
    centroids[:, 1],
    centroids[:, 2],
    marker="x",
    s=300,
    c="red",
    edgecolors="black",
    label="Centroids"
)

plt.title(
    "Customer Segmentation using K-Means"
)

plt.xlabel("Monthly Charges")

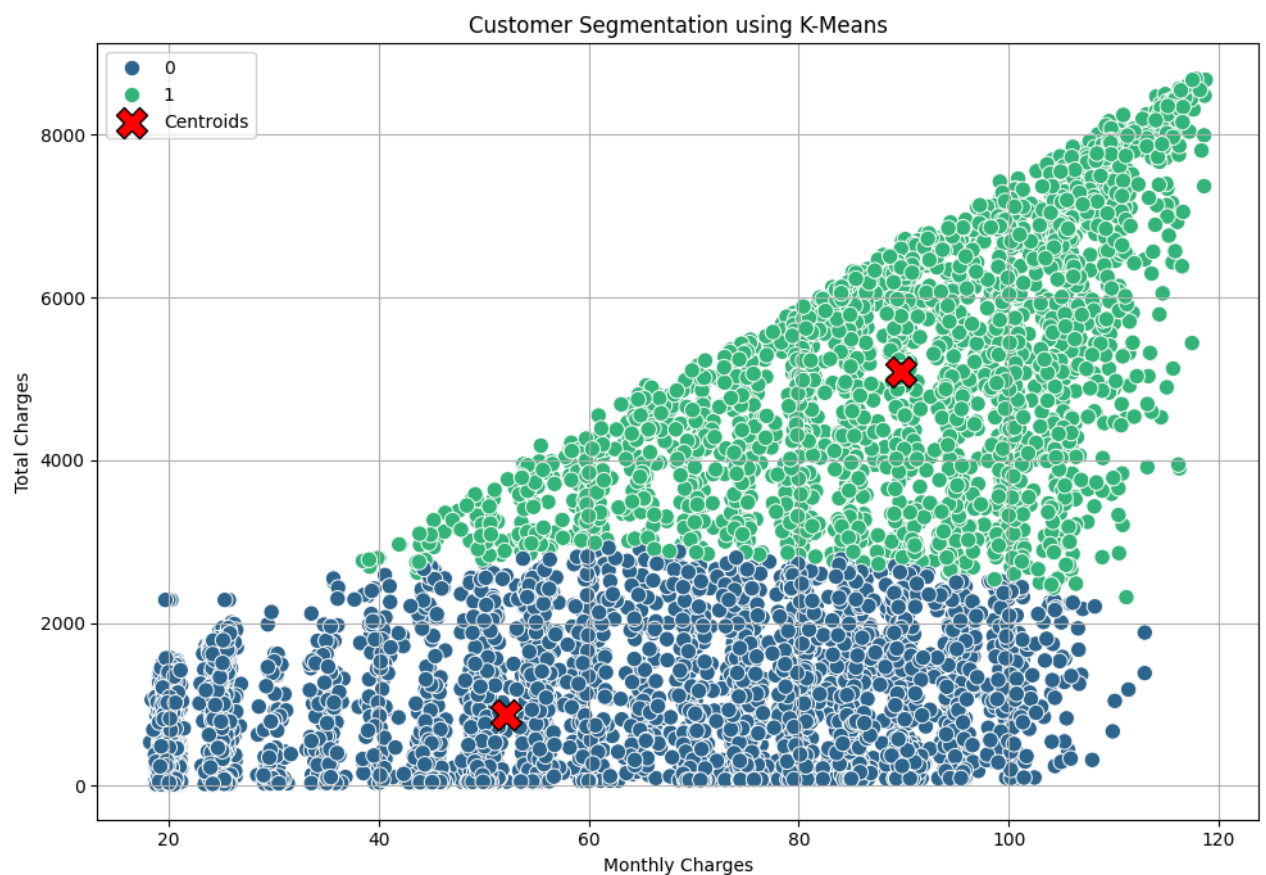
plt.ylabel("Total Charges")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()
print("Himani Chaudhary")
print("CU240250967")
```



Himani Chaudhary
CU240250967

```
# 16. TENURE VS MONTHLY CHARGES
# =====

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x="tenure",
    y="MonthlyCharges",
    hue="Cluster",
    palette="viridis",
    s=80
)

plt.title(
    "Tenure vs Monthly Charges by Customer Cluster"
)

plt.xlabel("Tenure (Months)")

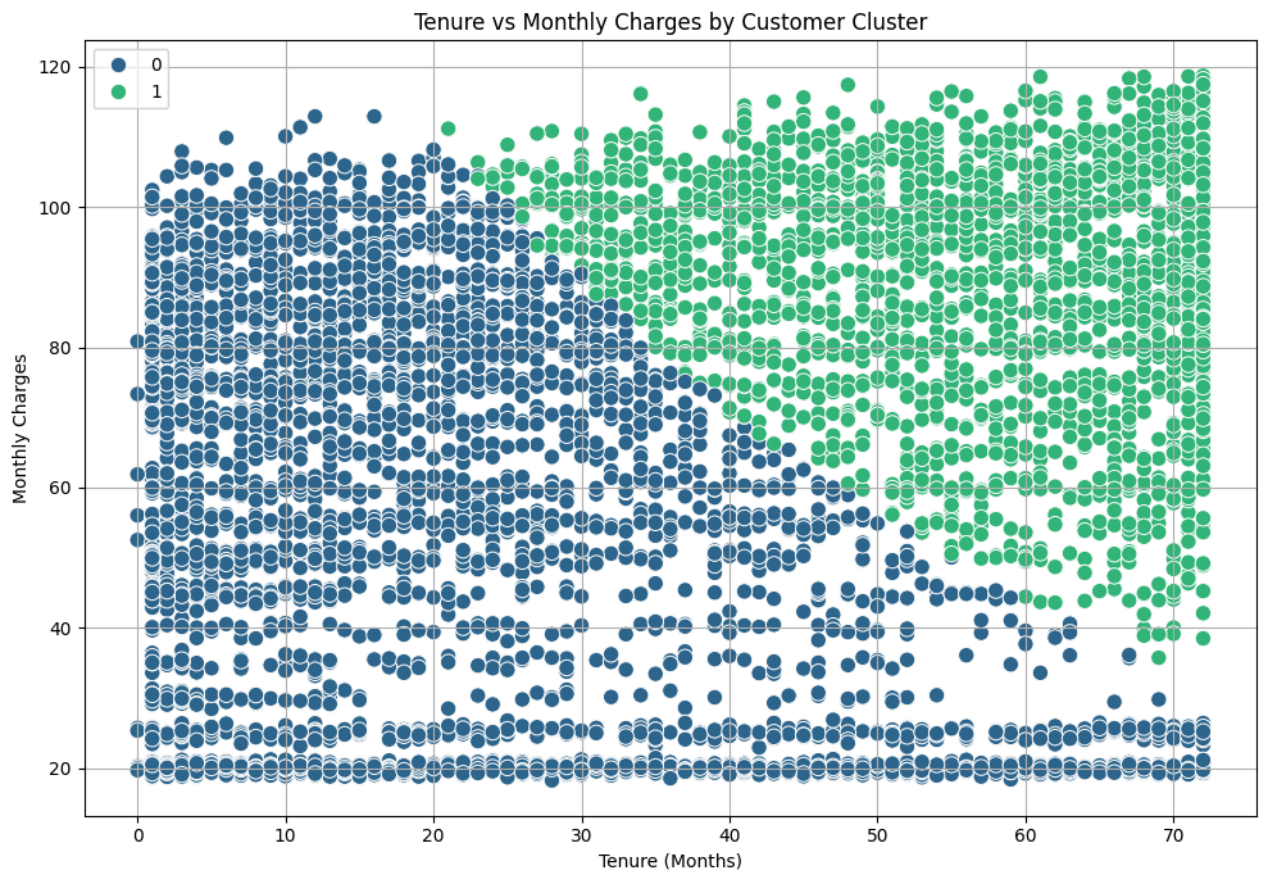
plt.ylabel("Monthly Charges")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()
print("Himani Chaudhary")
print("CU240250967")
```



Himani Chaudhary
CU240250967

```
# 17. CLUSTER SUMMARY BAR GRAPH
# =====

cluster_summary.plot(
    kind="bar",
    figsize=(10, 6)
)
```

```

plt.title(
    "Average Customer Features by Cluster"
)

plt.xlabel("Cluster")

plt.ylabel("Average Value")

plt.xticks(rotation=0)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.7
)

plt.tight_layout()

plt.show()
print("Himani Chaudhary")
print("CU240250967")

```



Himani Chaudhary
CU240250967

```

# 18. SAVE RESULT
# =====

# Save result as a NEW CSV file.
# Do NOT overwrite the original dataset.

output_file = "/content/WA_Fn-UseC_-Telco-Customer-Churn.csv"

# Save result
df.to_csv(
    output_file,
    index=False
)

print("\n" + "=" * 60)
print("FINAL RESULT")
print("=" * 60)

print("HIMANI CHAUDHARY")

print("\nClustering completed successfully!")

print("\nOptimal Number of Clusters:",
      best_k)

print("\nResult saved at:")

```

```
print(output_file)

print("\n" + "=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)
print("Himani Chaudhary")
print("CU240250967")
```